

Universidad Nacional Autónoma de México

Facultad de Ciencias

Simulación de algoritmos de planificación de CPU

Algoritmo Round Robin

20 de septiembre de 2026

1. Introducción

La planificación de procesos es una de las funciones fundamentales de un sistema operativo. Cuando existen varios procesos que requieren utilizar el procesador el sistema operativo debe determinar cuál de ellos recibirá el recurso y durante cuánto tiempo.

Los algoritmos de planificación de CPU establecen diferentes estrategias para organizar el acceso de los procesos al procesador. Entre los algoritmos existentes se encuentra Round Robin, este algoritmo es especialmente utilizado en sistemas donde se busca proporcionar una distribución equitativa del tiempo de CPU entre los procesos.

En este caso implementamos el algoritmo *Round Robin*. Para analizar su comportamiento realizamos simulaciones utilizando diferentes valores de *quantum*, en particular utilizamos $q = 2$, $q = 4$ y $q = 6$.

El programa va a recibir como entrada un conjunto de procesos y va a hacer uso de una cola circular para administrar aquellos procesos que se encuentran listos para utilizar el procesador. Durante la simulación se registraron diferentes métricas que permiten analizar el comportamiento de nuestro algoritmo.

2. Algoritmo Round Robin

Round Robin es un algoritmo de planificación, donde dentro del sistema operativo se asigna a cada proceso una porción de tiempo equitativo y ordenada, donde tratamos a los procesos con la misma prioridad, este fue diseñado especialmente para sistemas de uso compartido.

Vamos a definir un intervalo de tiempo al que le llamaremos quantum, cuya duración va a variar dependiendo del sistema, y la cola de procesos la vamos a estructurar como una cola circular. El planificador la va a recorrer asignando un quantum de tiempo a cada proceso, la organización de nuestra cola va a ser FIFO.

Los procesos listos para ejecutarse se mantienen en una cola. Y el proceso que se encuentra al frente de la cola recibe la CPU durante un máximo de q unidades de tiempo.

Si el proceso termina durante este intervalo, se elimina de la cola. En cambio, si todavía necesita tiempo de CPU, es colocado nuevamente al final de la cola.

El funcionamiento general lo podemos resumir de la siguiente manera:

1. Se agregan a la cola los procesos que ya han llegado.
2. Se selecciona el proceso que se encuentra al frente de la cola.
3. El proceso se ejecuta durante un máximo de q unidades de tiempo.
4. Si el proceso termina, se registran sus métricas.
5. Si el proceso no termina, se coloca nuevamente al final de la cola.
6. Se agregan los procesos que hayan llegado durante la ejecución.
7. El procedimiento continúa hasta que todos los procesos hayan terminado.

El objetivo de utilizar diferentes valores de quantums es observar cómo cambia el comportamiento del algoritmo al modificar la duración del intervalo de ejecución.

3. Estructuras de datos utilizadas

Para la implementación utilizamos principalmente dos estructuras: una estructura para representar los procesos y una cola circular para administrar los procesos listos.

3.1. Estructura Process

Cada proceso se representa mediante una estructura que almacena información relacionada con su ejecución.

Los principales campos utilizados son:

- **id**: identificador del proceso.
- **arrival**: tiempo en el que llega el proceso.
- **burst**: cantidad de tiempo de CPU que necesita.
- **priority**: prioridad asignada al proceso.
- **remaining**: tiempo de CPU que todavía necesita el proceso.
- **start_time**: primer instante en el que el proceso recibe la CPU.
- **finish_time**: instante en el que termina.
- **waiting_time**: tiempo total que permaneció esperando.
- **turnaround**: tiempo transcurrido desde su llegada hasta su finalización.
- **response_time**: tiempo transcurrido desde su llegada hasta la primera vez que recibe la CPU.

3.2. Cola circular

Para implementar Round Robin se utilizamos una cola circular.

La cola almacena los índices de los procesos que se encuentran listos para utilizar la CPU.

Se utilizaron tres variables principales:

- **frente**: indica la posición del proceso que será atendido.
- **final**: indica la posición donde se insertará el siguiente proceso.
- **cantidad**: indica cuántos procesos se encuentran actualmente en la cola.

La característica circular se consigue mediante la operación:

$$\text{posición siguiente} = (\text{posición actual} + 1) \bmod MAX_PROCESOS$$

Esto permite reutilizar las posiciones iniciales del arreglo cuando se alcanza el final.

4. Implementación

El programa fue implementado en lenguaje C. Cada proceso se representa mediante cuatro datos:

$$ID, \quad arrival, \quad burst, \quad priority$$

Por ejemplo:

1	1	0	2	1
2	2	2	4	2
3	3	4	6	3

La prioridad se almacena dentro de la estructura del proceso debido a que forma parte del formato de entrada (en este caso). Sin embargo, en la simulación de Round Robin esta información no interviene en la selección del siguiente proceso, ya que la prioridad es la misma para todos.

4.1. Reinicio de las simulaciones

Debido a que el programa ejecuta Round Robin varias veces utilizando el mismo conjunto de procesos, antes de cada simulación tenemos que reiniciar las métricas.

En particular, se van a restablecer:

- tiempo restante;
- tiempo de inicio;
- tiempo de finalización;
- tiempo de espera;
- tiempo de retorno;
- tiempo de respuesta.

Esto nos va a permitir que cada ejecución con un quantum diferente sea independiente de las ejecuciones anteriores.

5. Métricas utilizadas

Para analizar el comportamiento del algoritmo se utilizamos tres métricas principales.

5.1. Tiempo de retorno

El tiempo de retorno (*turnaround time*) corresponde al tiempo transcurrido desde que el proceso llega hasta que termina:

$$T_{\text{retorno}} = T_{\text{finalización}} - T_{\text{llegada}}$$

5.2. Tiempo de espera

El tiempo de espera representa el tiempo durante el cual el proceso permaneció esperando para utilizar la CPU.

Se calcula mediante:

$$T_{espera} = T_{retorno} - T_{burst}$$

donde T_{burst} representa el tiempo total que el proceso efectivamente utiliza la CPU.

5.3. Tiempo de respuesta

El tiempo de respuesta corresponde al intervalo entre la llegada del proceso y la primera vez que recibe la CPU:

$$T_{respuesta} = T_{inicio} - T_{llegada}$$

Esta métrica es especialmente importante en sistemas interactivos, pues nos permite observar cuánto tiempo debe esperar un proceso antes de comenzar a recibir atención del procesador.

6. Datos de entrada

Para realizar las pruebas se utilizó un conjunto de 100 procesos.

Los procesos se almacenaron en un archivo de texto y cada línea contiene cuatro valores:

ID	Arrival	Burst	Priority
----	---------	-------	----------

Los tiempos de llegada avanzan en intervalos de dos unidades y los tiempos de CPU utilizados por los procesos siguen el patrón:

2, 4, 6, 8, 10

La prioridad también sigue un patrón de valores entre 1 y 5. (pero en este caso RR no lo utiliza para decidir que proceso pasa primero).

Un ejemplo de las primeras líneas del archivo es:

1	1	0	2	1
2	2	2	4	2
3	3	4	6	3
4	4	6	8	4
5	5	8	10	5
6	6	10	2	1

El archivo completo contiene 100 procesos.

7. Compilación y ejecución

Para compilar el programa utilizamos el compilador GCC.

El comando empleado fue:

```
1 gcc -Wall -Wextra -std=c11 simuladorPlanProcesos.c -o simulador
```

Donde:

- `-Wall`: activa un conjunto amplio de advertencias del compilador.
- `-Wextra`: activa advertencias adicionales.
- `-std=c11`: indica que se utilizará el estándar C11.
- `-o simulador`: establece el nombre del archivo ejecutable.

Una vez compilado, podemos ejecutar el programa mediante:

```
1 ./simulador < Procesos.txt
```

También se puede guardar la salida en un archivo:

```
1 ./simulador < Procesos.txt > resultados.txt
```

De esta manera se obtiene un archivo que contiene los intervalos de ejecución y los resultados correspondientes a cada quantum. (en nuestro caso los resultados están en el archivo resultados.txt)

8. Prueba de funcionamiento

El programa realiza tres simulaciones consecutivas utilizando el mismo conjunto de 100 procesos.

Las simulaciones corresponden a:

1. Round Robin con $q = 2$.
2. Round Robin con $q = 4$.
3. Round Robin con $q = 6$.

En cada ejecución se muestran los intervalos en los cuales cada proceso utiliza la CPU. Por ejemplo, la salida comienza mostrando información de la forma:

```
1 Tiempo    0 -    2 : P1
2 Tiempo    2 -    6 : P2
3 Tiempo    6 -   10 : P3
4 ...
```

Posteriormente se muestran los resultados individuales de cada proceso y finalmente los promedios.

9. Análisis de resultados

A partir de los resultados obtenidos (resultados.txt) se puede observar que el cambio del quantum modifica de manera importante las métricas de planificación.

9.1. Tiempo de espera

El tiempo de espera promedio obtenido fue:

$$250.16 \rightarrow 240.64 \rightarrow 230.86$$

al pasar de $q = 2$ a $q = 4$ y posteriormente a $q = 6$.

Para el conjunto de procesos utilizado en esta práctica, el promedio disminuyó conforme aumentó el quantum.

9.2. Tiempo de retorno

El comportamiento del tiempo de retorno promedio fue:

$$256.16 \rightarrow 246.64 \rightarrow 236.86$$

para $q = 2$, $q = 4$ y $q = 6$, respectivamente.

Al igual que el tiempo de espera, el tiempo de retorno promedio disminuyó en las simulaciones realizadas al aumentar el quantum.

9.3. Tiempo de respuesta

En contraste con las métricas anteriores, el tiempo de respuesta promedio aumentó:

$$71.52 \rightarrow 122.82 \rightarrow 157.12$$

al incrementar el quantum.

Esto muestra que, para el conjunto de datos utilizado, el cambio del quantum produce un comportamiento diferente dependiendo de la métrica que estemos analizando.

10. Comparación de los diferentes quantums

La Tabla ?? permite comparar directamente las tres ejecuciones.

Cuadro 1: Comparación de los resultados obtenidos.

Quantum	Espera	Retorno	Respuesta
2	250.16	256.16	71.52
4	240.64	246.64	122.82
6	230.86	236.86	157.12

Los resultados nos muestran que no todas las métricas presentan el mismo comportamiento ante el cambio del quantum.

Para este conjunto específico de procesos, los tiempos promedio de espera y retorno disminuyeron al aumentar el quantum, mientras que el tiempo de respuesta promedio aumentó.

Por esta razón, el comportamiento de un algoritmo de planificación debe analizarse considerando las métricas relevantes para el sistema y no únicamente una de ellas.

11. Consideraciones de implementación

Durante la implementación se tomaron algunas decisiones importantes.

11.1. Uso de una cola circular

La utilización de una cola circular permite implementar de manera natural la política de Round Robin, ya que los procesos que no terminan después de consumir su quantum regresan al final de la cola.

La operación circular nos evita tener que desplazar todos los elementos del arreglo cada vez que extraemos un proceso.

11.2. Procesos que llegan durante la ejecución

Después de ejecutar cada proceso se revisa si existen nuevos procesos cuya llegada sea menor o igual al tiempo actual.

Estos procesos son agregados a la cola de procesos listos.

Por lo tanto, si un proceso nuevo llega exactamente al finalizar un quantum, se incorpora a la cola antes de volver a insertar el proceso que acaba de ser ejecutado.

12. Limitaciones

La simulación utiliza un arreglo con una capacidad máxima de:

$$MAX_PROCESOS = 200$$

Por lo tanto, el programa puede almacenar hasta 200 procesos.

Además, el algoritmo Round Robin utilizado en esta práctica no considera la prioridad para decidir qué proceso debe ejecutarse. Aunque la prioridad se encuentra almacenada en la estructura `Process`, esta información no interviene en la selección de procesos de Round Robin.

13. Conclusiones

La implementación nos permitió ejecutar el algoritmo con diferentes valores de quantum y obtener métricas relacionadas con el desempeño de los procesos.

Para el conjunto de 100 procesos utilizado, los resultados mostraron que al aumentar el quantum de 2 a 4 y posteriormente a 6, el tiempo de espera promedio pasó de 250.16 a 240.64 y finalmente a 230.86.

El tiempo de retorno promedio presentó un comportamiento similar, pasando de 256.16 a 246.64 y posteriormente a 236.86.

Por otra parte, el tiempo de respuesta promedio aumentó de 71.52 con $q = 2$, a 122.82 con $q = 4$ y 157.12 con $q = 6$.

Estos resultados nos permiten observar que modificar el quantum afecta de manera diferente a las distintas métricas. Por lo tanto, el comportamiento de Round Robin depende tanto del valor del quantum como de las características del conjunto de procesos que se está ejecutando.

Dado que en general este algoritmo teóricamente tiene un tiempo de espera relativamente grande, nos garantiza un reparto de la CPU entre todos los usuarios y arroja tiempo de respuesta bueno, podemos decir que nuestros resultados son satisfactorios.

14. Referencias

Referencias

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*.
- [2] Tanenbaum, A. S., & Bos, H. *Modern Operating Systems*.
- [3] Wikipedia. *Planificación Round-robin*. Wikipedia, la enciclopedia libre. Disponible en: https://es.wikipedia.org/wiki/Planificaci%C3%B3n_Round-robin [Consultado: 20 de septiembre de 2026].